

PROFEVIXO • GUÍA DE ESTUDIO

Complejidad computacional

Clases P y NP , reducciones.

Matemática discreta y fundamentos

Univ 4-5

20 EJERCICIOS

TEORÍA + RESULTADOS

Complejidad computacional

La **complejidad computacional** estudia los recursos (tiempo y memoria) que un algoritmo necesita en función del tamaño n de su entrada, y clasifica los problemas según la dificultad intrínseca de resolverlos. Este recurso cubre la notación asintótica, las clases P y NP , los verificadores, las reducciones, la NP -completitud y el problema abierto P vs NP .

Notación asintótica

La notación asintótica describe el crecimiento de una función cuando $n \rightarrow \infty$, ignorando constantes y términos menores.

Definición ($\mathcal{O}$). $f(n) = \mathcal{O}(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$0 \leq f(n) \leq c g(n) \quad \forall n \geq n_0$$

Es una cota **superior**: f crece a lo más como g .

Definición (Ω). $f(n) = \Omega(g(n))$ si existen $c > 0$ y n_0 tales que

$$f(n) \geq c g(n) \geq 0 \quad \forall n \geq n_0$$

Es una cota **inferior**: f crece al menos como g .

Definición (Θ). $f(n) = \Theta(g(n))$ si $f(n) = \mathcal{O}(g(n))$ y $f(n) = \Omega(g(n))$. Es una cota **ajustada**: mismo orden de crecimiento.

Definición (o). $f(n) = o(g(n))$ si

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Es decir, f crece estrictamente más lento que g .

Teorema (transitividad). Si $f = \mathcal{O}(g)$ y $g = \mathcal{O}(h)$, entonces $f = \mathcal{O}(h)$. Lo mismo vale para Ω y Θ .

Jerarquía de crecimiento.

$$\mathcal{O}(1) \subset \mathcal{O}(\log n) \subset \mathcal{O}(n) \subset \mathcal{O}(n \log n) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^3) \subset \mathcal{O}(2^n) \subset \mathcal{O}(n!)$$

Mini-ejemplo. $7n^2 + 3n + 5 = \Theta(n^2)$, porque el término dominante es n^2 y los demás son menores. En cambio $n = \mathcal{O}(n^2)$ pero $n^2 \neq \mathcal{O}(n)$.

Mini-ejemplo. $\log(n^3) = 3 \log n = \Theta(\log n)$: las constantes multiplicativas no cambian la clase.

Las clases P y NP

Trabajamos con **problemas de decisión**: dada una instancia, la respuesta es sí o no. Un problema se identifica con el lenguaje de las instancias cuya respuesta es sí.

Definición (P). P es la clase de los problemas de decisión que pueden resolverse en **tiempo polinomial** por una máquina de Turing determinista:

$$P = \bigcup_{k \geq 1} \text{DTIME}(n^k)$$

Son los problemas considerados **tratables**.

Definición (NP). NP es la clase de los problemas de decisión cuya respuesta afirmativa admite un **certificado** que se **verifica** en tiempo polinomial por una máquina determinista.

Equivalentemente, NP es la clase de problemas resolubles en tiempo polinomial por una máquina **no determinista**, que en cada paso puede "adivinar" la opción correcta.

Teorema. $P \subseteq NP$.

Demostración. Todo problema de P tiene un verificador que ignora el certificado y simplemente resuelve la instancia en tiempo polinomial. ■

Mini-ejemplo (en P). Ordenar una lista, hallar el camino más corto, decidir primalidad (test AKS), programación lineal y emparejamiento.

Mini-ejemplo (en NP). SAT, clique, viajante (versión de decisión), mochila (decisión) y coloración de grafos.

Observación. La pregunta «¿es $P = NP$?» es el problema abierto más famoso de la informática teórica.

Verificadores y certificados

Definición (verificador). Un lenguaje L está en NP si existe una relación $R(x, c)$ decidible en tiempo polinomial tal que

$$x \in L \iff \exists c \text{ con } |c| \leq \text{poly}(|x|) \text{ tal que } R(x, c) = 1$$

El testigo c se llama **certificado** y R es el **verificador**. La cota $|c| \leq \text{poly}(|x|)$ es esencial: un certificado exponencial no podría verificarse en tiempo polinomial.

Teorema (equivalencia de definiciones). $L \in NP$ (verificador polinomial) si y solo si L es aceptado por una máquina de Turing no determinista en tiempo polinomial.

Demostración (idea). Dado un verificador R , la máquina no determinista adivina c y evalúa $R(x, c)$. Recíprocamente, el certificado es la secuencia de elecciones no deterministas que lleva a la aceptación. ■

Mini-ejemplo (3-SAT). El certificado es una asignación de verdad de las variables. El verificador evalúa cada cláusula; el costo es lineal en el número de cláusulas, por tanto polinomial.

Mini-ejemplo (viajante de decisión). ¿Existe una ruta de costo $\leq k$ que visite todas las ciudades? El certificado es la ruta misma; verificar que es un recorrido válido y sumar sus costos toma tiempo polinomial.

Consecuencia. Si $P = NP$, todo problema verificable sería resoluble de forma eficiente y la criptografía clásica (por ejemplo RSA) quedaría comprometida.

Reducciones polinomiales

Definición (reducción polinomial). $A \leq_p B$ si existe una función f computable en tiempo polinomial tal que

$$x \in A \iff f(x) \in B$$

Es decir, transformamos instancias de A en instancias de B preservando la respuesta.

Teorema (preservación de la tratabilidad). Si $A \leq_p B$ y $B \in P$, entonces $A \in P$.

Demostración. Dado x , calcula $f(x)$ en tiempo polinomial (composición de polinomios es polinomial) y resuelve B sobre $f(x)$. La respuesta coincide con la de A . ■

Corolario (contrapositivo). Si $A \leq_p B$ y $A \notin P$, entonces $B \notin P$.

Mini-ejemplo. El camino más largo se reduce al camino más corto negando los pesos: una instancia de "camino de longitud $\geq k$ " se transforma en una de "camino de costo $\leq -k$ ". La transformación es polinomial.

Advertencia sobre la dirección. Para probar que B es difícil se reduce un problema **conocido difícil** A a B ($A \leq_p B$), no al revés.

NP-completitud y el teorema de Cook-Levin

Definición (NP-duro). B es **NP-duro** si todo $A \in NP$ cumple $A \leq_p B$.

Definición (NP-completo). B es **NP-completo** si:

1 $B \in NP$.

2 B es NP-duro: todo $A \in NP$ cumple $A \leq_p B$.

Teorema (propiedad clave). Si algún problema NP-completo pertenece a P , entonces $P = NP$.

Demostración. Sea B NP-completo con $B \in P$. Para todo $A \in NP$ se tiene $A \leq_p B$; por la preservación de la tratabilidad, $A \in P$. Luego $NP \subseteq P$ y, como $P \subseteq NP$, se concluye $P = NP$. ■

Método para probar NP-completitud.

1 Muestra que $B \in NP$ exhibiendo un certificado y un verificador polinomial.

2 Reduce un problema NP-completo **conocido** A a B : $A \leq_p B$.

Teorema de Cook-Levin (1971). SAT es NP-completo.

Idea de la demostración. Cualquier cómputo de una máquina de Turing no determinista que corre en tiempo polinomial se codifica como una fórmula proposicional gigante: sus variables describen la configuración (estado, contenido de la cinta y posición del cabezal) en cada instante. La fórmula es satisfacible si y solo si existe una computación aceptante. ■

Consecuencia. SAT es la primera piedra de la NP-completitud: mediante reducciones se demuestra que miles de problemas (3-SAT, clique, conjunto independiente, viajante, ...) también son NP-completos.

Problemas NP-completos famosos

- **SAT:** dada una fórmula proposicional, ¿existe una asignación que la haga verdadera?
- **3-SAT:** SAT restringido a cláusulas de exactamente tres literales.
- **Clique:** ¿contiene el grafo un subgrafo completo de tamaño k ?
- **Conjunto independiente:** ¿hay k vértices sin aristas entre ellos?
- **Cubrimiento de vértices:** ¿hay k vértices que toquen todas las aristas?
- **Coloración:** ¿es el grafo k -coloreable?
- **Camino hamiltoniano y viajante (decisión).**
- **Mochila (decisión):** ¿hay un subconjunto con peso $\leq W$ y valor $\geq V$?
- **Partición de conjuntos:** ¿puede dividirse un multiconjunto en dos partes de igual suma?
- **Cubrimiento exacto:** incluye rompecabezas como los tetrominós y el sudoku generalizado.

Clases vecinas. coNP contiene los complementos de los problemas de NP (por ejemplo, las tautologías); PSPACE agrupa los problemas resolubles con memoria polinomial y contiene a NP; EXP es la clase de tiempo exponencial. Se conoce

$$P \subseteq NP \subseteq PSPACE \subseteq EXP$$

y al menos una de las inclusiones es estricta, aunque no se sabe cuál.

Mini-ejemplo (clique vs conjunto independiente). En el grafo complemento $\bar{G}$, un clique de G es exactamente un conjunto independiente de $\bar{G}$. Construir $\bar{G}$ requiere tiempo polinomial, así que clique $\leq_p$ conjunto independiente.

P vs NP y estrategias para problemas duros

El problema P vs NP. ¿Existe un algoritmo polinomial para SAT? No se sabe. El Clay Mathematics Institute lo incluye entre los siete **problemas del milenio**.

Si $P = NP$. La optimización combinatoria sería eficiente, la criptografía clásica caería y gran parte de la matemática podría automatizarse.

Si $P \neq NP$. Existen problemas intrínsecamente difíciles y la seguridad criptográfica tiene una base sólida.

Estrategias prácticas ante problemas NP-duros.

- 1 **Aproximación:** algoritmos con garantía, como la 2-aproximación para el cubrimiento de vértices.

- 2 **Parametrización (FPT):** explotar un parámetro pequeño; por ejemplo, decidir si hay un cubrimiento de tamaño k en tiempo $\mathcal{O}(2^k \cdot n)$.
- 3 **Heurísticas:** búsqueda local, recocido simulado, algoritmos genéticos.
- 4 **SAT solvers:** en la práctica resuelven instancias industriales enormes mediante propagación y aprendizaje de cláusulas.

Mini-ejemplo (heurística). Para el cubrimiento de vértices, elegir siempre el vértice de mayor grado es una 2-aproximación: como mucho duplica el óptimo.

Ejercicios (20)

Nivel 1 · Básico

- 1 Escribe con cuantificadores la definición de $f(n) = \mathcal{O}(g(n))$.
- 2 Clasifica $6n^2 + 4n + 9$ en notación Θ .
- 3 ¿Es verdadera o falsa la afirmación $n = \mathcal{O}(\log n)$? Justifica.
- 4 Da la complejidad temporal de la búsqueda lineal en un arreglo de tamaño n .
- 5 Ordena de menor a mayor crecimiento: $n!$, $n \log n$, $\mathcal{O}(1)$, 2^n , n^2 .

Nivel 2 · Intermedio

- 6 Resuelve $T(n) = 2T(n/2) + n$ aplicando el teorema maestro.
- 7 Enuncia la definición de NP mediante un verificador.
- 8 Da un certificado y describe el verificador para la mochila de decisión.
- 9 Explica por qué $P \subseteq NP$.
- 10 ¿Cuál es la diferencia entre una reducción cualquiera y una reducción polinomial?

Nivel 3 · Avanzado

- 11 Define formalmente la reducción polinomial $A \leq_p B$.

12 Enuncia las dos condiciones que debe cumplir B para ser NP-completo.

13 Si $A \leq_p B$ y A es NP-duro, ¿qué se concluye sobre B ?

14 Enuncia el teorema de Cook-Levin y una de sus consecuencias.

15 Describe la idea de la reducción de clique a conjunto independiente.

Nivel 4 · Reto

16 Demuestra que si un problema NP-completo está en P , entonces $P = NP$.

17 Explica por qué el problema de la parada no es NP-completo.

18 Resuelve $T(n) = 4T(n/2) + n$ con el teorema maestro.

19 ¿Por qué el certificado debe tener longitud polinomial? Relaciónalo con la definición de NP.

20 Demuestra que $\mathcal{O}(n \log n) \subseteq \mathcal{O}(n^2)$.

Respuestas

1 Existen $c > 0$ y n_0 con $0 \leq f(n) \leq c g(n)$ para todo $n \geq n_0$.

2 $\Theta(n^2)$.

3 Falsa: n crece más rápido que $\log n$.

4 $\mathcal{O}(n)$.

5 $\mathcal{O}(1) < n \log n < n^2 < 2^n < n!$.

6 Con $a = 2, b = 2: n^{\log_b a} = n$, y $f(n) = n = \Theta(n)$; es el caso 2, luego $T(n) = \Theta(n \log n)$.

7 $L \in NP$ si existe una relación $R(x, c)$ decidible en tiempo polinomial con $x \in L \iff \exists c R(x, c) = 1$ y $|c| \leq \text{poly}(|x|)$.

8 El certificado es el subconjunto de ítems; el verificador comprueba en tiempo polinomial que el peso total no supera W y que el valor total alcanza V .

- 9 Todo algoritmo determinista polinomial es un caso particular del no determinista (que no adivina); o bien el verificador puede ignorar el certificado y resolver la instancia directamente.
- 10 Toda reducción transforma instancias preservando la respuesta; la polinomial exige además que la transformación se calcule en tiempo polinomial.
- 11 $A \leq_p B$ si existe f computable en tiempo polinomial con $x \in A \iff f(x) \in B$.
- 12 Que $B \in \text{NP}$ y que todo $A \in \text{NP}$ cumpla $A \leq_p B$.
- 13 B es NP-duro; si además $B \in \text{NP}$, entonces B es NP-completo.
- 14 SAT es NP-completo (1971); de él se derivan por reducción miles de problemas NP-completos.
- 15 En el grafo complemento, cada clique de G es un conjunto independiente, y viceversa; construir el complemento es polinomial.
- 16 Sea B NP-completo con $B \in \text{P}$. Para todo $A \in \text{NP}$, $A \leq_p B$; resolver B en tiempo polinomial resuelve A en tiempo polinomial, luego $\text{NP} \subseteq \text{P}$. Como $\text{P} \subseteq \text{NP}$, se tiene $\text{P} = \text{NP}$.
- 17 La parada es indecidible, mientras que NP solo contiene problemas decidibles; como todo lenguaje de NP es decidable, la parada queda fuera de NP .
- 18 $a = 4, b = 2$: $n^{\log_b a} = n^2$ y $f(n) = n = \mathcal{O}(n^{2-\epsilon})$; es el caso 1, luego $T(n) = \Theta(n^2)$.
- 19 El verificador debe leer el certificado dentro de su tiempo polinomial; si el certificado fuera exponencial, la verificación no podría ser polinomial y el problema no estaría en NP .
- 20 Para $n \geq 1$ se tiene $n \log n \leq n \cdot n = n^2$; con $c = 1$ y $n_0 = 1$ se cumple la definición de $\mathcal{O}(n^2)$.